Cryptography Unit II: Confidentiality and Modular Arithmetic (Detailed Notes)

Syllabus: Confidentiality using conventional encryption, traffic confidentiality, key distribution, random number generation, introduction to groups, rings and fields, modular arithmetic, Euclidean and extended Euclidean algorithms, prime numbers, Fermat's and Euler's theorems. Primary text: Atul Kahate, Cryptography and Network Security.

Part A: Confidentiality Using Conventional Encryption

1. Where to place the encryption

If encryption is used to protect data in a network, the first decision is where to apply it. There are two basic choices, link encryption and end-to-end encryption.

1.1 Link encryption

- Each vulnerable communication link gets an encryption device at both ends, so everything that crosses the link is encrypted, including headers.
- A message passing through several nodes is decrypted and re-encrypted at every node, using a different key for each link. The node therefore holds the message in plaintext while it processes it.
- All traffic, including routing information, is hidden on the wire, which gives a degree of traffic-flow confidentiality.
- Drawbacks: every intermediate switch, router or gateway must be trusted and physically secure, and many pairs of devices need keys. A compromise of any node exposes the traffic passing through it.

1.2 End-to-end encryption

- Encryption is performed at the source and decryption at the destination. The data stays encrypted as it crosses every node.
- In a packet-switched network, only the user data can be encrypted. The header must stay readable so that nodes can route the packet.
- Advantage: intermediate nodes never see plaintext, so a compromised node does not reveal content. Keys are shared only between the two end systems.
- Drawback: an attacker watching the network can still see who talks to whom and how much (traffic analysis), because the headers are visible.

Point	Link encryption	End-to-end encryption
Where applied	Hop by hop, on each link	Source to destination
Plaintext exposure	At every intermediate node	Only at the two end systems

Point	Link encryption	End-to-end encryption
Headers	Encrypted	In clear (needed for routing)
Keys	One per link, shared by the two nodes on it	One per pair of communicating end systems or users
Who must be trusted	All intermediate nodes	Only the end systems
Traffic analysis	Partly prevented	Not prevented
Transparent to user	Yes, done by the network	User or host must take part

Best practice is to use both. End-to-end encryption protects the content from source to destination, and link encryption hides the headers and the traffic pattern on each link. Application layer end-to-end encryption can also be used, where the application encrypts only the sensitive fields (selective-field confidentiality).

2. Traffic confidentiality

Even with encrypted content, an attacker who monitors the network can learn:

- the identities of the partners,
- how frequently they communicate,
- message pattern, length or quantity, which may hint at activity (for example a sudden rise in traffic before an event),
- events that correlate with specific conversations.

Countermeasures:

- **Link encryption** hides source and destination addresses in headers on each link. Within a node they are still readable, so nodes must be secured.
- **Traffic padding** produces ciphertext output continuously, even without plaintext. When no real data is present, random data (dummy traffic) is encrypted and sent. This makes the observed traffic level constant, so the attacker cannot tell real traffic from padding.
- **End-to-end padding** by padding message lengths to a fixed size and sending cover messages, so that length and pattern leak less.
- **Routing control**: avoiding paths or nodes that are untrusted.

3. Key distribution

The strength of a conventional encryption system depends on keeping the key secret. Two parties A and B that wish to communicate must share a secret key, and they must change it frequently so that a compromised key reveals only a little data. Key distribution is the means of delivering a key to two parties without letting others see it.

3.1 Four ways to deliver a key

1. A selects a key and physically delivers it to B.
2. A third party selects a key and physically delivers it to A and B.
3. If A and B have previously used a key, one party transmits the new key to the other

encrypted with the old key.
4. If A and B each have an encrypted connection to a third party C, C delivers a key on the encrypted links to A and B.

Options 1 and 2 are practical only for link encryption, where the devices are physically close. For end-to-end encryption across a large network option 4 is used, as option 3 would give an attacker the new key as well if one key were ever exposed.

3.2 Number of keys

If N users each need to talk securely to every other, the number of pairwise keys is N(N - 1) / 2. For 1,000 users that is 499,500 keys. A key distribution center reduces this to one master key per user.

3.3 Key hierarchy

- **Session key**: a temporary key used to encrypt data for one logical connection (a session). It is discarded after the session. Because it is used on large amounts of data, it must be changed often.
- **Master key (permanent key)**: a long-term key shared between a user (or node) and the key distribution center. It is used only to encrypt session keys for distribution. Master keys must be distributed by a non-cryptographic means such as hand delivery.

3.4 Key Distribution Center (KDC) scenario

The KDC shares a unique master key with each user. A (initiator) wants a session key to communicate with B. Let K_a and K_b be the master keys of A and B, ID_A and ID_B their identifiers, and N_1, N_2 nonces.

1. A to KDC: ID_A, ID_B, N_1. A asks for a session key for B and includes a nonce so the reply can be matched to this request.
2. KDC to A: $E(K_a, [K_s, ID_B, N_1, E(K_b, [K_s, ID_A])])$. A decrypts this and checks that the nonce N_1 and ID_B match its request. This confirms the reply is fresh and not a replay.
3. A to B: $E(K_b, [K_s, ID_A])$. A forwards the part meant for B (called a ticket). Only B can read it.
4. B to A: $E(K_s, N_2)$. B shows it holds the session key and sends a fresh nonce.
5. A to B: $E(K_s, f(N_2))$, where f is a simple function such as $N_2 + 1$. This convinces B that the message in step 3 is not a replay.

Steps 4 and 5 are an authentication handshake. After step 5 A and B use K_s to protect their session.

3.5 Hierarchical key control

For very large networks a single KDC is not practical. Local KDCs serve nearby users and talk to a global KDC when two users in different domains need a key. This limits the effort of master key distribution and makes the system more reliable, but it creates a chain of trust that must be secured.

3.6 Session key lifetime

The more often session keys change, the more secure the system, because an opponent has less ciphertext for a given key. But key changes cost time and add overhead. For a connection-oriented protocol a new key is used for each connection. For a connectionless protocol, a new key is used after a fixed number of messages or after a fixed time.

3.7 Decentralized key control

A fully decentralized scheme needs every end system to hold master keys with all others, N(N - 1) / 2 in total. This is workable only in small networks. The protocol is: A sends B a request and a nonce N_1. B replies with a message encrypted under their shared master key containing the session key, both identifiers, $f(N_1)$ and a new nonce N_2. A returns $E(K_s, f(N_2))$.

4. Random Number Generation

4.1 Why cryptography needs random numbers

- Session keys and master keys must be unpredictable.
- Nonces in authentication protocols prevent replay.
- Public-key algorithms need random primes (RSA key generation) and random values (ElGamal, signatures).
- Initialization vectors for block cipher modes, salts for password hashing, and the key stream of a one-time pad all depend on randomness.

4.2 Requirements

- **Randomness**: the sequence should show uniform distribution (each value roughly equally frequent) and independence (no value can be inferred from others). These properties are checked by statistical tests.
- **Unpredictability**: it must be computationally infeasible to predict the next value from earlier ones (forward unpredictability) or to work out earlier values from the present state (backward unpredictability). This matters more than statistical quality in cryptography.

4.3 True and pseudorandom generators

Point	TRNG (true random number generator)	PRNG (pseudorandom number generator)
Source	A physical process: thermal noise, radioactive decay, clock jitter, keystroke timing	A deterministic algorithm started from a seed
Output	Non-deterministic, not reproducible	Fully determined by the seed, so reproducible
Speed	Slow	Fast
Use	Seeding and generating long-term keys	Bulk random data, expanded from a good seed

A cryptographically secure PRNG (CSPRNG) is a PRNG whose output cannot be distinguished from random or predicted without knowing the seed. Modern systems use a TRNG to seed a CSPRNG (for example the NIST SP 800-90A designs such as CTR_DRBG, or the operating system's random device).

4.4 Linear congruential generator (LCG)

$X_{n+1} = (a \cdot X_n + c) \bmod m$, with modulus m, multiplier a, increment c and seed X_0. The choice of a, c and m decides the period. The Hull-Dobell conditions for full period m are: c and m are relatively prime, a - 1 is divisible by all prime factors of m, and a - 1 is divisible by 4 if m is. Example: a = 5, c = 3, m = 16, $X_0 = 7$ gives 7, 6, 1, 8, 11, 10, 5, 12, 15, 14, 9, 0, 3, 2, 13, 4, then 7 again, a full period of 16 values.
LCGs are fast and statistically good, but they are predictable. Given a few outputs, the parameters and the seed can be recovered, so they must not be used for cryptographic keys.

4.5 Blum Blum Shub (BBS) generator

1. Choose two large primes p and q, both congruent to 3 mod 4, and let $n = p \cdot q$.
2. Choose a seed s that is relatively prime to n. Set $X_0 = s^2 \bmod n$.
3. For each step: $X_i = (X_{i-1})^2 \bmod n$. The output bit B_i is the least significant bit of X_i.

Small example: p = 11, q = 23, n = 253, s = 3. $X_0 = 9$. $X_1 = 81$, $X_2 = 6561 \bmod 253 = 236$, $X_3 = 236^2 \bmod 253 = 36$, $X_4 = 1296 \bmod 253 = 31$. The output bits are the least significant bits: 1, 0, 0, 1.
BBS is slow but cryptographically secure under the assumption that factoring n is hard. It is "unpredictable to the left" and "to the right" as long as the factors are secret. The real parameters use primes of hundreds of digits.

Part B: Mathematics for Cryptography

5. Algebraic Structures: Group, Ring, Field

5.1 Group

A group $(G, \cdot)$ is a set with one binary operation satisfying:

1. **Closure**: for all a, b in G, $a \cdot b$ is in G.
2. **Associativity**: $(a \cdot b) \cdot c = a \cdot (b \cdot c)$.
3. **Identity**: there is an element e with $a \cdot e = e \cdot a = a$.
4. **Inverse**: for each a there is a^{-1} with $a \cdot a^{-1} = e$.

- An **abelian group** is also commutative: $a \cdot b = b \cdot a$.
- The **order** of a group is the number of elements. The order of an element a is the smallest k with $a^k = e$.
- A **cyclic group** has a **generator** g such that every element is a power of g.
- Examples: the integers under addition; $Z_n = \{0, 1, ..., n - 1\}$ under addition mod n; Z_n^* (numbers in 1..n-1 coprime to n) under multiplication mod n, which has order $\varphi(n)$. For n = 10, $Z_{10}^* = \{1, 3, 7, 9\}$, and 3 is a generator because 3, 9, 7, 1 are its successive powers.

5.2 Ring

A ring (R, +, ×) is a set with two operations such that:

- (R, +) is an abelian group (with identity 0).
- Multiplication is closed and associative.
- The distributive laws hold: a(b + c) = ab + ac and (a + b)c = ac + bc.

A **commutative ring** also has ab = ba. An **integral domain** is a commutative ring with a multiplicative identity (1) and no zero divisors (ab = 0 implies a = 0 or b = 0). Example: the integers. Z_n is a commutative ring for every n. It is an integral domain only when n is prime.

5.3 Field

A field (F, +, ×) is an integral domain in which every non-zero element has a multiplicative inverse. Addition, subtraction, multiplication and division (except by zero) all work.

- Infinite fields: the rationals, the reals, the complex numbers.
- Finite fields (Galois fields) exist only for orders p^n, where p is a prime. They are written $GF(p^n)$.
- **GF(p)**: the integers mod a prime p. Example, inverses in GF(7): 1→1, 2→4, 3→5, 4→2, 5→3, 6→6.
- **$GF(2^n)$**: polynomials with coefficients in GF(2), with arithmetic done modulo an irreducible polynomial of degree n. Addition is XOR. AES works in $GF(2^8)$ with the irreducible polynomial $x^8 + x^4 + x^3 + x + 1$.

Example in $GF(2^3)$ with the irreducible polynomial $x^3 + x + 1$: multiply $(x^2 + 1)$ by $(x^2 + x + 1)$. The product is $x^4 + x^3 + x + 1$. Reduce using $x^3 = x + 1$ and $x^4 = x^2 + x$: the result is $(x^2 + x) + (x + 1) + x + 1 = x^2 + x$.
Example in $GF(2^8)$: {57} + {83} = {d4} (XOR of the bytes), and {57} · {83} = {c1}.

5.4 Hierarchy

Every field is an integral domain, every integral domain is a commutative ring, and every ring is an abelian group under addition. Z_n is a field if and only if n is prime.

6. Modular Arithmetic

6.1 Definitions

- For integer a and positive integer n, a mod n is the remainder r ($0 \le r < n$) when a is divided by n: $a = qn + r$.
- $a \equiv b \pmod n$ (a is congruent to b modulo n) means n divides a - b, that is a mod n = b mod n.
- Congruence modulo n is an equivalence relation. The classes are the residue classes [0], [1], ..., [n - 1], and Z_n is the set of residues.

6.2 Properties

- (a + b) mod n = [(a mod n) + (b mod n)] mod n

- $(a - b) \bmod n = [(a \bmod n) - (b \bmod n)] \bmod n$
- $(a \times b) \bmod n = [(a \bmod n) \times (b \bmod n)] \bmod n$
- Addition and multiplication are commutative, associative and distributive mod n.
- Additive inverse of a: n - a (for $a \neq 0$). Every element of Z_n has one.
- Multiplicative inverse of a exists if and only if gcd(a, n) = 1. If n is prime, every non-zero a has an inverse.
- The cancellation law $ab \equiv ac \pmod n$ implies $b \equiv c \pmod n$ holds only if gcd(a, n) = 1.

6.2.1 Example: modular exponentiation by square and multiply

Compute $7^{13} \bmod 11$. Write $13 = 8 + 4 + 1$. $7^2 = 49 \equiv 5$, $7^4 \equiv 5^2 = 25 \equiv 3$, $7^8 \equiv 3^2 = 9$. So $7^{13} \equiv 9 \times 3 \times 7 = 189 \equiv 2 \pmod{11}$. Check with Fermat: $7^{10} \equiv 1$, so $7^{13} \equiv 7^3 = 343 \equiv 2$.
This technique keeps numbers small by reducing after every multiplication and is how RSA exponentiation is done.

7. Euclidean Algorithm

The greatest common divisor gcd(a, b) is the largest integer that divides both. The Euclidean algorithm uses $\gcd(a, b) = \gcd(b, a \bmod b)$ with $\gcd(a, 0) = a$.
Example: gcd(1970, 1066).
- $1970 = 1 \times 1066 + 904$
- $1066 = 1 \times 904 + 162$
- $904 = 5 \times 162 + 94$
- $162 = 1 \times 94 + 68$
- $94 = 1 \times 68 + 26$
- $68 = 2 \times 26 + 16$
- $26 = 1 \times 16 + 10$
- $16 = 1 \times 10 + 6$
- $10 = 1 \times 6 + 4$
- $6 = 1 \times 4 + 2$
- $4 = 2 \times 2 + 0$

The last non-zero remainder is 2, so gcd(1970, 1066) = 2.
Shorter example: gcd(252, 105): $252 = 2 \times 105 + 42$; $105 = 2 \times 42 + 21$; $42 = 2 \times 21 + 0$. So gcd = 21. The number of steps is proportional to the number of digits, so the method is fast even for huge numbers.

8. Extended Euclidean Algorithm

It finds integers x and y such that $ax + by = \gcd(a, b)$ (Bézout's identity). If gcd(a, n) = 1, then $ax + ny = 1$, which gives $a x \equiv 1 \pmod n$, so x is the multiplicative inverse of a.
Method: run the Euclidean algorithm, then substitute backwards, expressing each remainder as a combination of a and b.
Example 1: inverse of 11 mod 26.
- $26 = 2 \times 11 + 4$
- $11 = 2 \times 4 + 3$
- $4 = 1 \times 3 + 1$

$1 = 4 - 3 = 4 - (11 - 2 \times 4) = 3 \times 4 - 11 = 3(26 - 2 \times 11) - 11 = 3 \times 26 - 7 \times 11$. So $11^{-1} \equiv -7 \equiv 19 \pmod{26}$. Check: $11 \times 19 = 209 = 8 \times 26 + 1$.

Example 2: inverse of 23 mod 120.

- $120 = 5 \times 23 + 5$
- $23 = 4 \times 5 + 3$
- $5 = 1 \times 3 + 2$
- $3 = 1 \times 2 + 1$

$1 = 3 - 2 = 3 - (5 - 3) = 2 \times 3 - 5 = 2(23 - 4 \times 5) - 5 = 2 \times 23 - 9 \times 5 = 2 \times 23 - 9(120 - 5 \times 23) = 47 \times 23 - 9 \times 120$. So $23^{-1} \equiv 47 \pmod{120}$. Check: $23 \times 47 = 1081 = 9 \times 120 + 1$.
This is exactly how the RSA private exponent d is computed from e and φ(n).

9. Prime Numbers

- A prime is an integer greater than 1 whose only positive divisors are 1 and itself. 2 is the only even prime.
- A composite number is greater than 1 and not prime.
- Two integers a and b are **relatively prime** (coprime) if gcd(a, b) = 1. Distinct primes are always coprime, but coprime numbers need not be prime (8 and 15).

9.1 Fundamental theorem of arithmetic

Every integer greater than 1 can be written as a product of primes in exactly one way, apart from the order. Example: $360 = 2^3 \times 3^2 \times 5$. The gcd is found by taking the smaller power of each common prime, and the lcm by taking the larger.

9.2 Finding primes

- **Sieve of Eratosthenes**: list numbers up to N, repeatedly take the smallest unmarked number as prime and cross out its multiples. Primes up to 30: 2, 3, 5, 7, 11, 13, 17, 19, 23, 29.
- There are infinitely many primes (Euclid). The number of primes up to N is roughly N / ln N (prime number theorem). So randomly chosen large numbers have a good chance of being prime, and RSA key generation simply tests random candidates.

9.3 Primality testing

- Trial division works only for small numbers.
- **Fermat test**: if $a^{n-1} \not\equiv 1 \pmod{n}$, n is composite. Passing the test does not prove primality. Carmichael numbers such as 561 pass for every base coprime to n.
- **Miller-Rabin test** (probabilistic, used in practice): write $n - 1 = 2^k m$ with m odd. Pick a random a. Compute $b = a^m \bmod n$. If $b \equiv \pm 1$, n is probably prime. Otherwise square b up to k - 1 times. If −1 appears, n is probably prime; if not, n is definitely composite. Each round gives at least 3/4 chance of catching a composite, so about 40 independent rounds make the error negligible.

10. Fermat's Little Theorem

If p is prime and a is an integer not divisible by p, then $a^{p-1} \equiv 1 \pmod{p}$. An equivalent form holds for every integer a: $a^p \equiv a \pmod{p}$.
Proof idea: the numbers a, 2a, ..., (p - 1)a are all distinct mod p and non-zero, so they are a

rearrangement of 1, 2, ..., p - 1. Multiplying them gives a^{p-1} (p - 1)! ≡ (p - 1)! (mod p). Since (p - 1)! is coprime to p, it can be cancelled.
Uses: fast modular exponentiation with large exponents, modular inverse $a^{-1} \equiv a^{p-2}$ (mod p), and primality testing.
Example 1: 3^{100} mod 7. $3^6 \equiv 1$, 100 = 16 × 6 + 4, so $3^{100} \equiv 3^4 = 81 \equiv 4$.
Example 2: 2^{100} mod 13. $2^{12} \equiv 1$, 100 = 8 × 12 + 4, so $2^{100} \equiv 2^4 = 16 \equiv 3$.
Example 3: inverse of 5 mod 13 is 5^{11} mod 13. $5^2 = 25 \equiv 12 \equiv -1$, so $5^4 \equiv 1$ and $5^{11} = 5^8 \times 5^2 \times 5 \equiv 1 \times (-1) \times 5 = -5 \equiv 8$. Check: 5 × 8 = 40 = 3 × 13 + 1.

11. Euler's Totient Function and Euler's Theorem

11.1 Euler's totient φ(n)

φ(n) is the number of integers in 1..n that are relatively prime to n.

- φ(1) = 1.
- For a prime p: φ(p) = p - 1.
- For a prime power: $\varphi(p^k) = p^k - p^{k-1}$.
- Multiplicative: if gcd(m, n) = 1 then φ(mn) = φ(m) φ(n). So for distinct primes p and q, φ(pq) = (p - 1)(q - 1).
- General formula: φ(n) = n × product over each distinct prime p dividing n of (1 - 1/p).

Examples: φ(10) = φ(2) φ(5) = 1 × 4 = 4 (the numbers 1, 3, 7, 9). φ(35) = 4 × 6 = 24. φ(36) = 36 × (1/2) × (2/3) = 12. φ(100) = 100 × (1/2) × (4/5) = 40.

11.2 Euler's theorem

For every a and n with gcd(a, n) = 1: $a^{\varphi(n)} \equiv 1$ (mod n). Fermat's theorem is the special case n = p, where φ(p) = p - 1.
Proof idea: the same as Fermat's, using the φ(n) residues coprime to n instead of 1..p - 1.
Examples:

- $3^{\varphi(10)} = 3^4 = 81 \equiv 1$ (mod 10).
- 7^{222} mod 10: φ(10) = 4, 222 mod 4 = 2, so $7^{222} \equiv 7^2 = 49 \equiv 9$.
- Inverse by Euler: $a^{-1} \equiv a^{\varphi(n)-1}$ (mod n). Inverse of 3 mod 10 is $3^3 = 27 \equiv 7$. Check: 3 × 7 = 21 ≡ 1.

11.3 Link to RSA

RSA picks n = pq, with φ(n) = (p - 1)(q - 1), and e, d with ed ≡ 1 (mod φ(n)). Then for a message M coprime to n, $M^{ed} = M^{1 + k\varphi(n)} \equiv M$ (mod n), which is why decryption recovers the message.

12. Likely Exam Questions and Answer Outlines

1. **Link encryption versus end-to-end encryption.** Use the table in Part A, and conclude that both are used together.
2. **Explain traffic analysis and the ways to prevent it.** Describe what the attacker learns, then link encryption, traffic padding and routing control.
3. **Explain key distribution using a KDC.** Draw the five-step message flow in Section 3.4 with master keys and session key, and explain the role of nonces.

4. **Differentiate TRNG and PRNG. Explain Blum Blum Shub.** Give the table, the three steps and the example.
5. **Define group, ring and field with examples.** Give the axioms in order and the hierarchy, and state that Z_n is a field when n is prime.
6. **Find gcd and modular inverse using the Euclidean and extended Euclidean algorithms.** Show every division step and the back-substitution, then verify.
7. **State and prove Fermat's theorem. Use it to find a remainder.**
8. **Define φ(n). State Euler's theorem and use it.**

Sources consulted

- William Stallings, Cryptography and Network Security (chapters on confidentiality, key distribution, random numbers, finite fields and number theory), and Atul Kahate, Cryptography and Network Security (course text).
- NIST FIPS 197 for the GF(2^8) arithmetic and the AES polynomial.